



Integrated Chatbot User Experience for the JD Edwards Administration

May 7, 2020 | Version 1.01
Copyright © 2020, Oracle and/or its affiliates
Confidential: Public

PURPOSE STATEMENT

To introduce the basic concepts of the Digital Assistant and give hands-on experience of how to build a Digital Assistant that can be embedded within a web or mobile app for enabling stakeholders to use the same process to build their own Digital Assistant for JD Edwards.

DISCLAIMER

The following is intended to outline our general product direction. It is intended for information purposes only, and may not be incorporated into any contract. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described for Oracle's products remains at the sole discretion of Oracle.

TABLE OF CONTENTS

Purpose Statement	1
Disclaimer	1
OVERVIEW	3
Project	3
Use Case	3
The JD Edwards Solution	3
Demo Script or Interaction	3
Benefits	4
Minimum Technical Requirements	4
Digital Assistant	5
What Are Skills?	5
Overview of Skill Development	5
Create Skill	5
Which Training Model to Use?	8
Flows	8
Testing	10
Custom Component	11
How Do Custom Components Work?	11
The Component Service	11
Adding Custom Component as an Embedded Container	16
Adding Custom Component to the Dialog Flow	17
Adding Skill to Digital Assistant (Optional)	19
Channels	21
Embedding a Chatbot in a JD Edwards E1 Page	21
Prerequisites	21
Steps	21
Uploading Chatbot as an E1Page	26



OVERVIEW

Project

This case study highlights a possible JD Edwards feature as part of which a Digital Assistant would be built to manage JD Edwards administration. JD Edwards Advanced Development Team is currently researching the use of Oracle Digital Assistant within EnterpriseOne. This functionality is not generally available.

Use Case

There are certain actions JD Edwards administrators perform often. Sometimes these actions involve updates to multiple applications to complete the workflow. Creating users is one such use case which involves the following steps:

1. Add user record in Address Book.
2. Create a user profile.
3. Set up a role.
4. Assign environments to the role.
5. Assign the role to a user.
6. Set up security for the user.

Using the Oracle Digital Assistant and orchestrations, this action can be performed quickly through chat. A similar model, the integrated modern chatbot interface technology, can be applied to several business processes within JD Edwards. Creating users includes several repetitive steps, which are prone to errors when performed manually. To perform these steps, the chatbot interface could open a multi-step process using a progressive conversational interaction. Here, the user focuses only on inputs and the system executes the repetitive steps on the back end.

The JD Edwards Solution

At a high level, the solution includes:

1. Chatbot interface for user interaction and experience, using Oracle Digital Assistant (ODA)
2. JD Edwards orchestrations to automate repetitive tasks

Demo Script or Interaction

This is an example of a conversation using a chatbot (Edward) to automate the user creation process:

Admin: Create user John

Edward: What is the address book number? (Format: AB# 1234)

Admin: AB# 1007

Edward: What is the alpha name? (Format: Financial Distribution Company)

Admin: 'John D'

{After few seconds}

Edward: User creation successful

Benefits

- Quicker turnaround time for business process completion
- Improved productivity and reduced error rates, with the system performing the repetitive tasks, enabling the user to focus on exceptions and complex business conditions

Minimum Technical Requirements

1. JDE Edwards Enterprise One 9.2 and above
2. Oracle Digital Assistant instance—This is where the Digital Assistant Skill (chatbot) is built
3. Node JS—To install all the required modules for the package

DIGITAL ASSISTANT

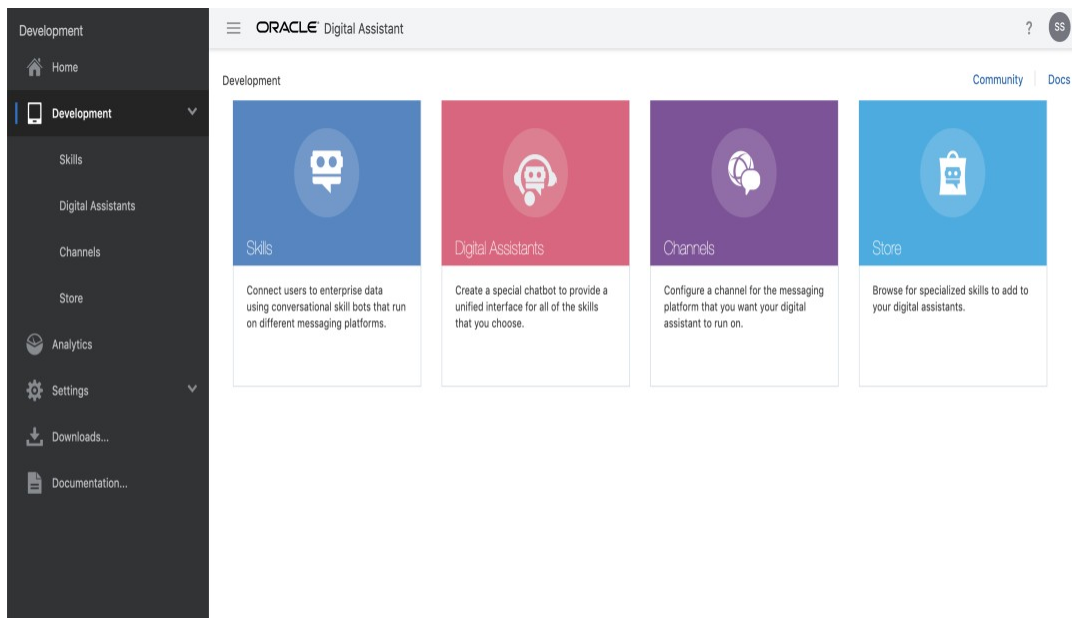
What Are Skills?

A Skill is nothing but a chatbot that completes a task through a combination of text messages and simple UI elements such as select lists. While a Skill or chatbot is a component that can perform a particular task, a Digital Assistant is a combination of multiple Skills with the ability to route the request from the user to the correct Skill to perform the required task. A Digital Assistant can facilitate tasks through enterprise messaging; however, it is not a replacement for a mobile or web app. It instead provides a new channel.

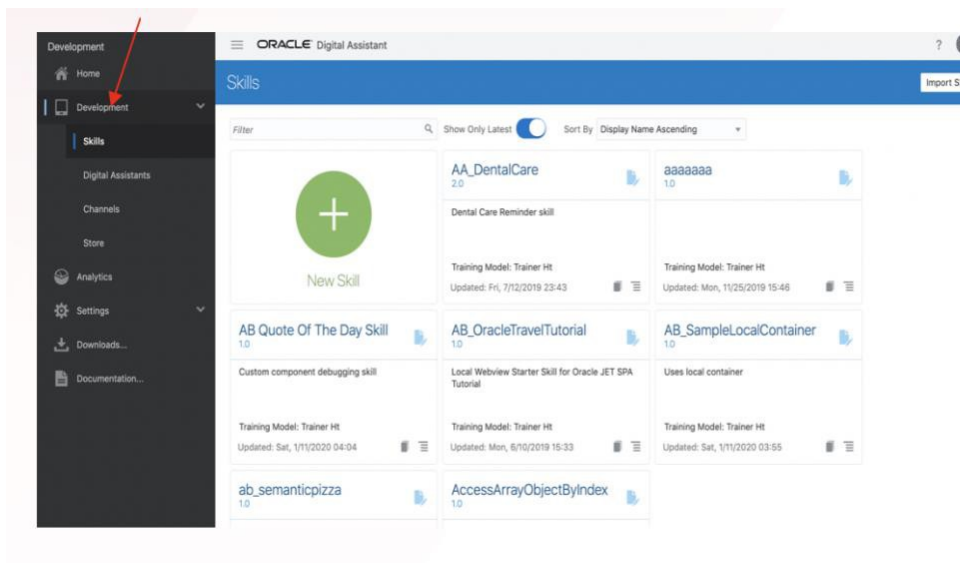
Overview of Skill Development

Create Skill

1. Open your Oracle Digital Assistant (ODA) instance and click the Menu icon  to open the side menu.



2. Select Development and then Skills to see the existing Skills and also to create a new Skill.



3. To create a new Skill or chatbot (Skill and chatbot will be used interchangeably in this document), click the New Skill button. Next, you need to enter some basic information, such as the name and a brief description of the Skill. In this particular case, we will create a very simple Skill for a JD Edwards administrator. The Skill will provide the JD Edwards administrator with the ability to create users with commands.

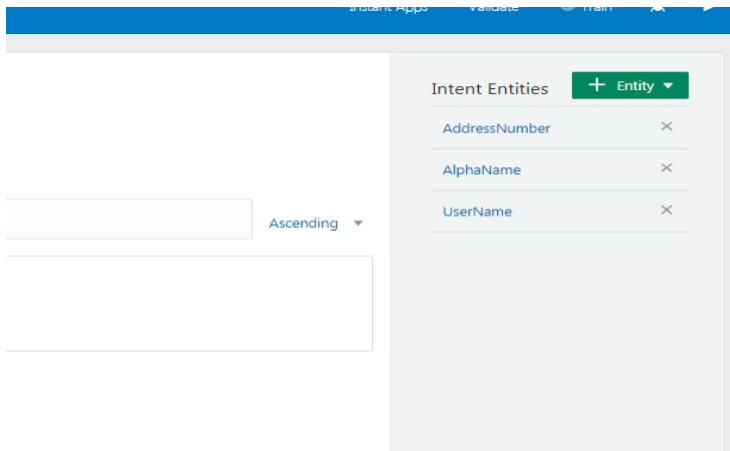
A screenshot of the 'Create Skill' dialog box. It features a green circle with a white plus sign and the text 'New Skill' on the left. The right side contains a form with the following fields: 'Display Name' (with a note: 'Will be used in the Skill Catalog. This value can't be modified later.'), 'Name' (with a note: 'The official identifier for the skill. This value can't be modified later.'), 'Version' (set to '1.0'), and 'One-Sentence Description'. A green 'Create' button is located at the bottom right.

4. **Create Intents:** The first option within the Skill is called Intents, which are categories of actions or tasks you expect your Skill to perform. Here, we will record some questions or utterances about the topic so that the Skill can acquire the information it needs to fulfill the requirement. Click the **Intent** button to create your first intent, and name it **Create User**. Now we can create different utterances or statements to help the Skill to understand what the user needs. We shall add three utterances for our use case:
 - create user JDE with AB# 1234
 - create new user
 - create user

5. **Create Entities:** Entities are variables that identify key pieces of information from the user input that enable the Skill to fulfill a task. Go to the Entities tab and create the following entities that are required to create a user:

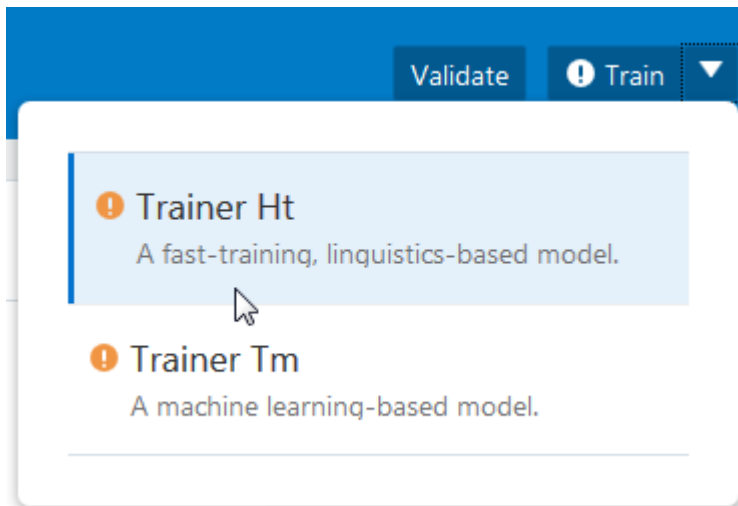
Entity Name	Type	Value
UserNameRegex	Regular Expression	[a-zA-Z][a-zA-Z0-9]{2,7}
UserName	Derived	UserNameRegex
AddressNumber	Regular Expression	[aA][Bb]# [0-9]{1,6}
AlphaName	Regular Expression	!.*!

6. **Add Entities:** Add the above created entities to the Create User intent you previously created.



7. **Train the Skill:** To enable your Skill to reference intents when it parses the user input, you need to train it. Specifically, you need to train it with the intents and their utterances (collectively, the training data), so that it can resolve the user input to one of the intents. Click Train, choose a model, and then click Submit.

Note: You must have a minimum of two intents to train the bot and each intent should have at least three utterances. You can create a dummy intent to meet this requirement if needed.



Which Training Model to Use?

- After you create the initial training corpus, you should start with Trainer Ht because it does not require a large set of utterances. As long as there are enough utterances to disambiguate the intents, your bot will be able to resolve the user input to an intent.
- Refine your corpus, and retrain with Trainer Ht. Repeat as necessary. Training is an iterative process.
- Use the trainer Train with Trainer Tm when you have accumulated a robust set of intents.

Flows

Next, by creating the dialog flow you need to give the Skill the ability to express its intelligence to its users. The dialog flow describes how your Skill or chatbot reacts as different intents are resolved. It defines what your chatbot says to its users, how it prompts them for input, and how it returns data. Think of the dialog flow as a flow chart that has been transposed to a simple markdown language. In Oracle Digital Assistant, this markdown language is a version of YAML called BotML.

By default, the following template will be available:

```

1 #metadata: information about the flow
2 # platformVersion: the version of the bots platform that this flow was written to work with
3 metadata:
4   platformVersion: 1.0
5 main: true
6 name: Fresh_Bot
7 #context: Define the variables which will used throughout the dialog flow here.
8 context:
9   variables:
10    #The syntax for defining the variables is variablename: "variableType".
11    # The "variableType" can be defined as a primitive type ("int", "string", "boolean"), "list", or an entity name. A variable can also hold the results returned by the Intent Engine. For these variables, the
12    # "variableType" must be "nlpresult" (for example, iResult: "nlpresult").
13    greeting: "string"
14    name: "string"
15 #states is where you can define the various states within your flow.
16 # The syntax for defining a state is
17 # statename:
18 #   component: Specify the component you want to use. This can be either a Built-In or custom component.
19 #   properties:
20 #     property1: "value" (These are the properties to the specified component
21 #   transitions: You can specify one of the following four
22 #     next: Specify the state you want to execute next after this state. By default it will execute the state specified after this
23 #     error: Specify the state you want to execute in case the component encounters any error in execution.
24 #   actions: You can handle various actions returned by the components here the syntax is actionName: "statename"
25 #     action1: statel
26 #     return: "done" You can exit the flow using the return statement
27 states:
28   askGreeting:
29     component: "System.List"
30     properties:
31       options: "Hello!, Ola!, Vannakam!, Namaste!"
32       prompt: "Hi there! What would you like me to echo back?"
33       variable: "greeting"
34   askName:
35     component: "System.Text"
36     properties:
37       prompt: "What is your name?"
38       variable: "name"
39   start:
40     component: "System.Output"
41     properties:
42       text: "${greeting.value} ${name.value}"
43     transitions:
44       return: "done"

```

Clear the existing content and paste the content inside the box below: **(Note: Indentation should be strictly followed.)**

metadata:

platformVersion: "1.0"

main: true

name: "JDEBotMainFlow"

context:

variables:

iResult: "nlpresult"

states:

intent:

component: "System.Intent"

properties:

variable: "iResult"

transitions:

actions:

Create User: "createuserflow"

unresolvedIntent: "unresolved"

createuserflow:

```

component: "System.Output"
properties:
  text: "Sorry I am not yet programmed to create user"
transitions:
  return: "createuserflow"
unresolved:
  component: "System.Output"
  properties:
    text: "Sorry I don't understand that question!"
  transitions:
    return: "unresolved"

```

The following are some of the variables included in the YAML:

The **iResult** variable contains information and values generated by the natural language process engine. It helps to determine the intents that trigger the execution of the flow. This variable takes its value from **System.Intent**, which represents the engine instance and the algorithm.

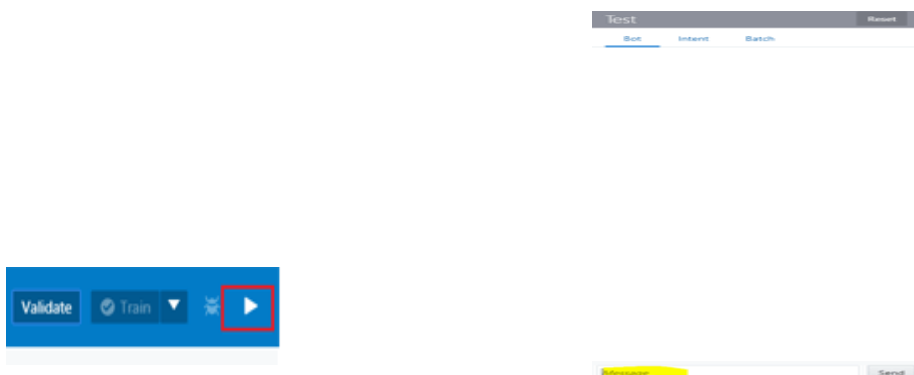
The **confidenceThreshold** variable is very important, as it defines the precision value that the engine uses to determine what is going to be executed when the intent has been evaluated. The engine and the algorithm evaluate each user intent and assign each a score; the score that is higher than the **confidenceThreshold** variable value is executed. Increasing or decreasing this value impacts the accuracy with which the bot can resolve the input from the user.

To validate the BotML, click the **Validate** button in the upper-right section of the Flow screen. You should see a confirmation message.

If there are two or more intents, then you need to train the bot. To do that, you need to click the Train button in the upper right of the screen so the bot can be trained with the Intents, Entities and Flow. Once you click Train, you are ready to test the bot. In this case, as you have only one intent you can skip training the bot.

Testing

Click the Play button, located in the upper-right part of the screen. Enter Create user. The system returns the message: "Sorry I am not yet programmed to create user". You get this message because the dialog flow is programmed to respond with this message for the create user intent.



CUSTOM COMPONENT

How Do Custom Components Work?

Your Skill uses custom components when it needs to return data, execute business logic, or render channel-specific UI components such as the carousel in Facebook Messenger.

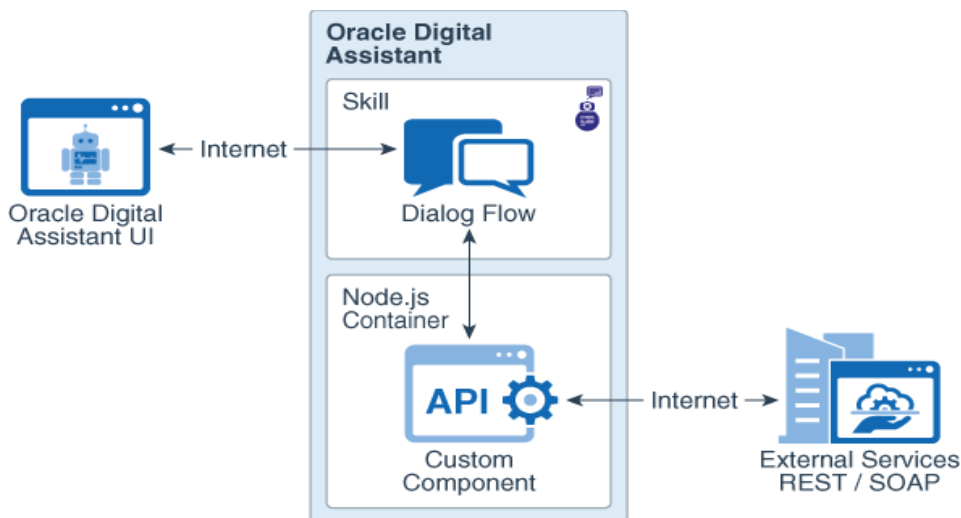
Similar to the built-in components, the custom components are reusable units of work that you define within the node of each state of your dialog flow. Unlike the built-in components, custom components perform actions that are specific to your Skill. They execute functions that the system components cannot execute.

Custom components do not reside within intelligent bots. Their functionality is provided through back-end services that are accessed through calls made to and returned from a REST service called the Component Service. As the Dialog Engine enters a state in the dialog flow, it assesses the component. When the Dialog Engine encounters one of the built-in components (prefixed with System.), it executes one of the generic tasks described in the built-in components: Properties, Transitions, and Usage. When the Dialog Engine discovers a custom component, however, it calls the Component Service, which hosts one or more custom components.

The Component Service first finds and then invokes the custom component on behalf of the Dialog Engine. When a custom component is invoked, it can pass input parameters to a back-end service and return the result. The Dialog Engine then resumes, moving on to the next state in the dialog flow, or to the state dictated by the action described in the returned JSON payload.

The Component Service

The Component Service is hosted in its own node container. In the following image, the node container is part of ODA, but it can be part of any other REST infrastructure as well.



To create your custom API, you must follow these steps:

1. You must first get the ODA Custom Component. You can get the component SDK from Oracle Technology Network's [Oracle Digital Assistant \(ODA\) and Oracle Mobile Cloud \(OMC\) Downloads](#) page. You can also access this page by clicking Downloads in the left navigation bar.

Oracle Digital Assistant (ODA) and Oracle Mobile Cloud (OMC) Downloads

This page contains all the downloads for Oracle Digital Assistant (ODA) and Oracle Mobile Cloud (OMC). OMC downloads are compatible with Autonomous Mobile Cloud Enterprise (AMCe).

ODA Custom Component SDKs

Module	Download	What's New
ODA Custom Component SDK 06/20/2019	 ODA Custom Component SDK	Oracle Digital Assistant custom components SDK and samples now hosted on GitHub.

- After you download and extract the component SDK file, you will have a folder named “bots-node-sdk-master”
- You should navigate to “../bots-node-sdk-master/examples/custom-components/starter/custom” which contains the following artifacts that you modify to build your service.

 **components** **spec** **.npmignore** **README.md** **main.js** **package.json**

4. Create a new javascript file named “create_user.js” within the components folder and copy the following snippet to the file. You can also delete the hello.world.js and resolveExpense.js within the components folder.

```
"use strict"

const rp = require('request-promise');

module.exports = {
  metadata: () => ({
    "name": "jde.CreateUser",    //component name
    "properties": {
      "userName": { "type": "string", "required": true },
      "addressNumber": { "type": "string", "required": true },
      "alphaName": { "type": "string", "required": true }
    },
    "supportedActions": [
    ]
  }),

  invoke: (conversation, done) => {
    var userName = conversation.properties().userName;
    var addressNumberStr = conversation.properties().addressNumber;
    // strip out prefix "AB# "
    var addressNumber = addressNumberStr.replace(/AB# /i, "");
    var alphaNameStr = conversation.properties().alphaName;
    // strip out single quotes
    var alphaName = alphaNameStr.replace(/'/g, "");
    conversation.logger().info('jde.CreateUser: creating JDE user ' + userName);
    var baseUrl = "https://machineip:port/jderest"; //add the url for your rest server
    var loginUrl = baseUrl + "/tokenrequest";
    //add user name and password to your rest server
    var options = {
      method: 'POST',
      uri: loginUrl,
      body: {
        username: "<user-name>",
        password: "<password>"
      },
      json: true // Automatically stringifies the body to JSON
    };
  }
};
```

```

rp(options)
.then(function (parsedBody) {
    var jsonData = JSON.stringify(parsedBody, null, '\t');
    var json = JSON.parse(jsonData);
    var token = json.userInfo.token;
    var orchUrl = baseUrl + "/orchestrator/ND_AddUser";
    var params = {};
    var inputs = []
    params.token = token;
    params.inputs = inputs;
    params.inputs.push({"name": "AddressNumber", "value": addressNumber});
    params.inputs.push({"name": "AlphaName", "value": alphaName});
    params.inputs.push({"name": "SearchType", "value": "E"});
    params.inputs.push({"name": "BusinessUnit", "value": "1"});
    params.inputs.push({"name": "UserID", "value": userName});
    params.inputs.push({"name": "Role", "value": "SYSADMIN"});
    params.inputs.push({"name": "DataSource", "value": "DEFAULT"});
    params.inputs.push({"name": "SystemUser", "value": "JDE"});
    params.inputs.push({"name": "Password", "value": "welc2jde"});
    var options = {
        method: 'POST',
        uri: orchUrl,
        body: params,
        json: true // Automatically stringifies the body to JSON
    };
    return rp(options);
})
.then((response) => {
    console.log('Response', response);
    conversation.reply({ text: 'User ' + userName + ' created successfully'});
    conversation.transition();
    done();
})
.catch(function (error) {
    console.log(error);
    conversation.reply({ text: 'User ' + userName + ' creation failed'});
    conversation.transition();
});

```



```

done();
});

}

};

) => {

```

This javascript file receives the entity values from the chat conversation and performs AIS calls to the orchestration ND_ADDUSER with the required inputs. The response received is then modified to a user readable form and sent as a reply to the conversation flow. Replace the values of highlighted text to values relevant for your JD Edwards environment.

The following table shows the service requests within the ND_ADDUSER orchestration and the tasks performed by these service requests:

SERVICE REQUESTS	INPUTS
Add user record	Addressumber, AlphaName, SearchType (E), BusinessUnit (1)
Create user profile	UserId, AddressNumver
Assign role to user	UserID, Role (SYSADMIN)
Set up security for user	UserID, DataSource (Default), SystemUser, Password

For more information on orchestrations, refer to the [JD Edwards EnterpriseOne Tools Orchestrator Guide](#).

5. Edit the package.json file to add the dependencies for the create_user.js file. Also, you can change the name of package.

```

{
  "name": "jde_admin",
  "version": "1.0.0",
  "description": "Simple Bots Custom Components in form of npm package",
  "main": "main.js",
  "dependencies": {
    "@oracle/bots-node-sdk": "^2.1.0",
    "request": "^2.88.2",
    "request-promise": "^4.2.5"
  },
  "scripts": {
    "bots-node-sdk": "bots-node-sdk",
    "start": "npm run bots-node-sdk -- service ."
  }
}

```

```

"prepack": "npm run bots-node-sdk -- pack --dry-run ."
},
"devDependencies": {
  "@oracle/bots-node-sdk": "^2.1.0"
}
}

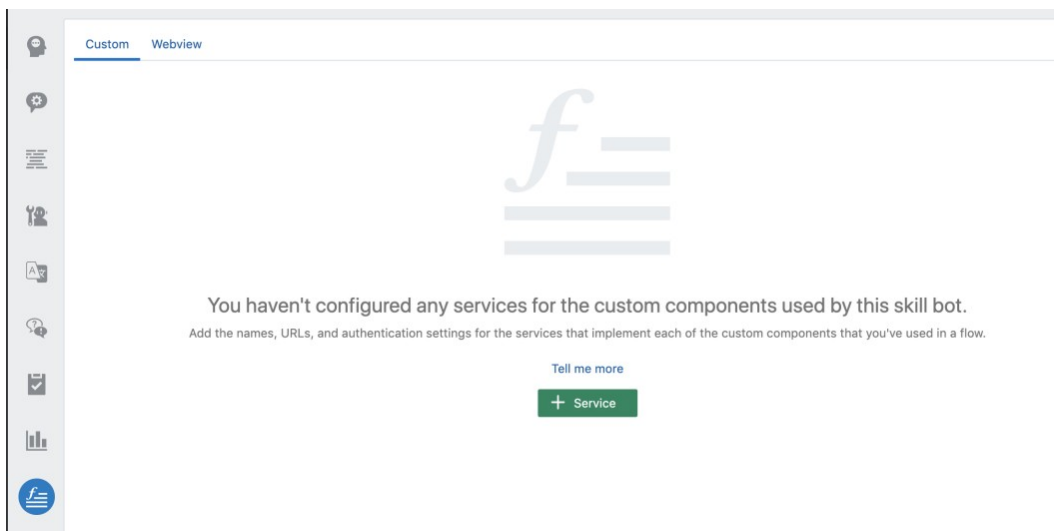
```

6. Install the node module. Navigate to the custom component file directory in the command line and run NPM install.
7. Run the bots-node-sdk pack from the custom component directory to create a .tgz file. The created .tgz file will have the name mentioned in the package.json file (jde_admin.tgz) .

Adding Custom Component as an Embedded Container

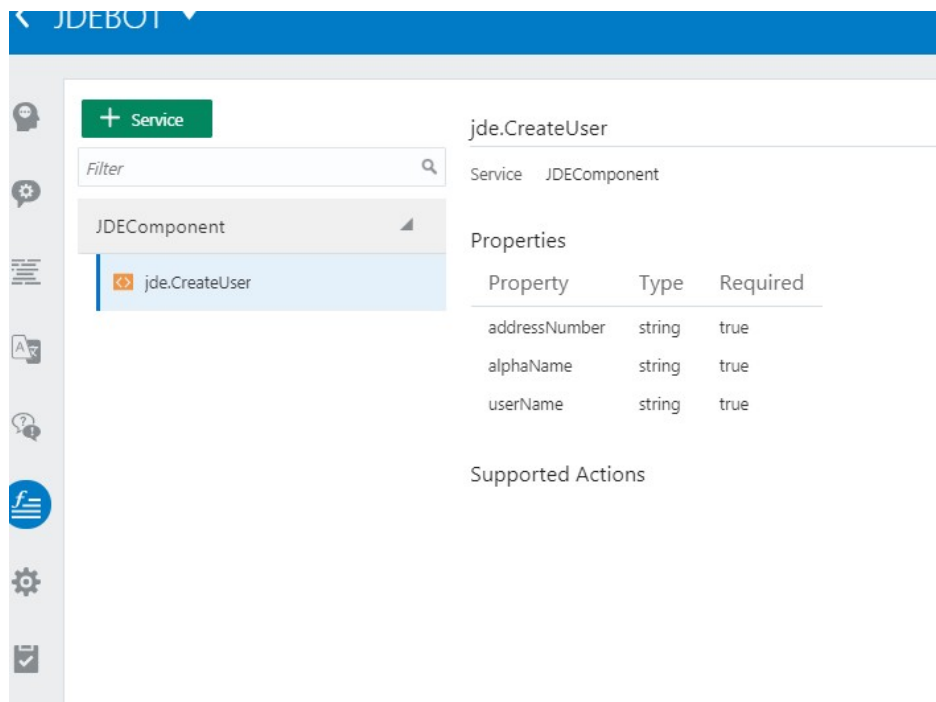
Navigate back to Development, Skills, your Skill (jde_admin) to add the customer component in the embedded container as a component service so that it can be discovered by the Dialog Engine.

1. In the left navigation bar, click Components.



2. Click Add Service to open the Create Service dialog box.

3. Add the name for the custom component service and as an optional step, you can add a description. Leave the Embedded container set in its default state and upload the custom component created in the previous task.



You are now ready to add the custom components to your OBotML definition. You can see your custom components loaded as shown above.

Adding Custom Component to the Dialog Flow

Navigate to Dialog Flow and replace the contents with the YAML code below. Also, replace the highlighted component name with your component name. Validate the YAML code with the Validate button in the upper-right section of the page. (**Note: Indentation should be strictly followed.**)

metadata:

platformVersion: "1.0"

main: true

name: "JDEdwardsBotMainFlow"

context:

variables:

iResult: "nlpresult"

userName: "UserName"

addressNumber: "AddressNumber"

alphaName: "AlphaName"

txnState: "string"

states:

intent:

```
component: "System.Intent"
properties:
  variable: "iResult"
transitions:
  actions:
    Create User: "startCreateUserFlow"
    unresolvedIntent: "unresolved"
startCreateUserFlow:
  component: "System.SetVariable"
  properties:
    variable: "userName"
    value: "${iResult.value.entityMatches['UserName']}[0]}"
  transitions: {}
setAddressNumber:
  component: "System.SetVariable"
  properties:
    variable: "addressNumber"
    value: "${iResult.value.entityMatches['AddressNumber']}[0]}"
  transitions: {}
setAlphaName:
  component: "System.SetVariable"
  properties:
    variable: "alphaName"
    value: "${iResult.value.entityMatches['AlphaName']}[0]}"
  transitions: {}
askUserName:
  component: "System.Text"
  properties:
    prompt: "What's the user name?"
    variable: "userName"
    maxPrompts: 1
  transitions:
    actions:
      cancel: "intent"
askAddressNumber:
  component: "System.Text"
  properties:
    prompt: "What's the Address Number (format AB# 1234)?"
```

```

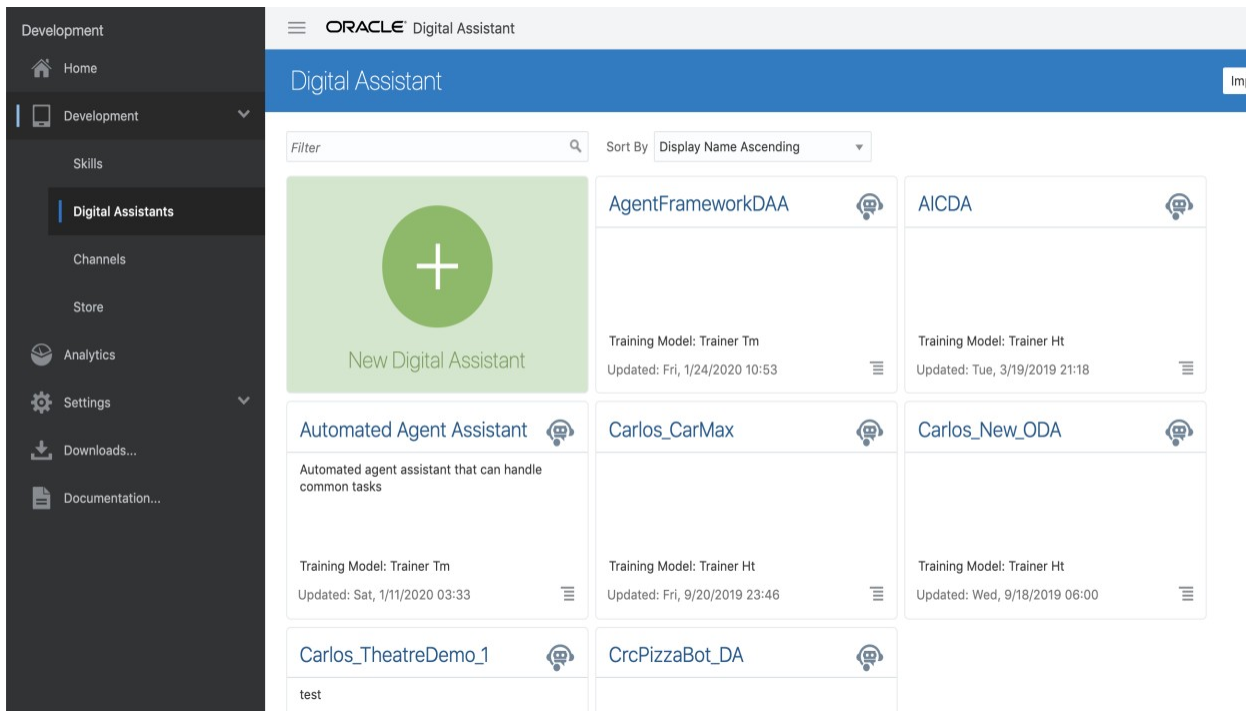
    variable: "addressNumber"
    maxPrompts: 1
  transitions:
    actions:
      cancel: "intent"
  askAlphaName:
    component: "System.Text"
    properties:
      prompt: "What's the AlphaName (format 'Financial Distribution
      Company')?" variable: "alphaName"
      maxPrompts: 1
    transitions:
      actions:
        cancel: "intent"
  execCreateUser:
    component: "jde.CreateUser"
    properties:
      userName: "${userName.value}"
      addressNumber: "${addressNumber.value}"
      alphaName: "${alphaName.value}"
    transitions:
      return: "execCreateUser"
  unresolved:
    component: "System.Output"
    properties:
      text: "Sorry I don't understand that question!"
    transitions:
      return: "unresolved"

```

Adding Skill to Digital Assistant (Optional)

You can use the Skill directly within the channel for routing or you can publish the Skill and add it to a Digital Assistant.

1. From the Development drop-down menu, select the Digital Assistants option in your instance.



- Click New Digital Assistant button. In the dialog box that is displayed, enter a Name and a Display Name for the Digital Assistant and click Create.

Create Digital Assistant

Display Name *

This value can't be modified later.

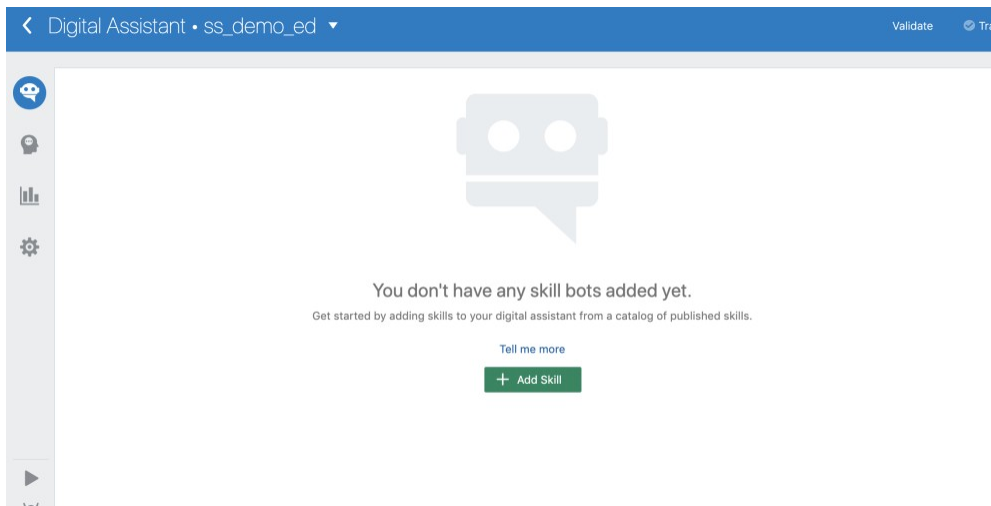
Name *

This value can't be modified later.

Description

Create

The newly created Digital Assistant is displayed as shown below. You can add the Skills to this Digital Assistant.



3. Click Add Skill and add the Skill created in the previous section. You can add multiple Skills to the DA in this way.

Channels

Digital Assistants or Skills are not apps that you download from an app marketplace. Instead, you access them through messaging platforms or through client messaging apps. Channels, which are platform-specific configurations, provide this access. A single DA or Skill can have several channels configured so that it can run on different services simultaneously.

1. Go to Development, Channels, and add a new Oracle Web channel. The system generates a channel ID that can be used to call the Digital Assistant service from the web.
2. Make sure that you route the channel to the Skill created in the above section and enable the channel.
3. If you have created a Digital Assistant for the Skill then you can route the channel to the Digital Assistant.

EMBEDDING A CHATBOT IN A JD EDWARDS E1 PAGE

Prerequisites

- To get the Oracle Web Client SDK, go to the Oracle Technology Network's [Oracle Digital Assistant \(ODA\) and Oracle Mobile Cloud \(OMC\) Downloads](#) and download Oracle Web SDK 20.3.x.
- Install Node.js on your machine by going here: <https://nodejs.org/en/download/>.

Steps

1. Extract Oda-native-client-sdk-js-20.3.x.zip.
2. Navigate to "/Oda-native-client-sdk-js-20.3.x/native-client-sdk-js/samples/web" directory.
3. Rename the index.html to home.html in the web directory.

4. Replace the contents of home.html with the following snippet (for customizing the UI and renaming the chatbot) removing the unwanted part of UI available in the sample.

```
<!DOCTYPE html>

<html lang="en-US" class="no-js">

<head>

  <title>Oracle | Integrated Cloud Applications and Platform Services</title>

  <meta name="Title" content="Oracle | Integrated Cloud Applications and Platform Services">

  <meta name="Description"

    content="Oracle offers a comprehensive and fully integrated stack of cloud applications and platform services.">

  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1">

  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">

  <link rel="shortcut icon" href="images/favicon.ico">

  <link rel="stylesheet" href="styles/redwood-base.css" rel="preload" as="style" onload="this.rel='stylesheet'"

    onerror="this.rel='stylesheet'">

<script>

  /**

   * Flag to set mechanism to connect to chat server

   * true - Use authentication token based mechanism

   * false - Use parameters to connect to chat server

   */

  var clientAuthEnabled = false;

</script>

<script src="scripts/settings.js"></script>

<script>

  /**

   * Initialize the SDK and set a name for it which will be used to

   * refer it, and call its APIs.

   */

  function initSdk(name) {

    // Default name is Bots

    if (!name) {

      name = 'Bots';

    }

    setTimeout(() => {

      let Bots;

      if (clientAuthEnabled) {
```



```

        Bots = new WebSDK(chatWidgetSettings, generateToken);
    } else {
        Bots = new WebSDK(chatWidgetSettings);
    }
    Bots.connect();

    window[name] = Bots;
});
}
</script>
<script src="scripts/web-sdk.js" onload="initSdk('Bots')"></script>
</head>
<body>
    <h1 align= center>JD Edwards Virtual Assistant</h1>
</div>
</body>
</html>

```

In the modified home.html file, the body of the original html file is replaced with the line “**<h1 align= center>JD Edwards Virtual Assistant</h1>**”

5. Save the below image to the directory “/Oda-native-client-sdk-js-20.3.x/native-client-sdk-js/samples/web/images” as edward.png



6. Replace the contents of settings.js located in the scripts directory with the following snippet:

```
'use strict';

if (typeof clientAuthEnabled === 'undefined') {
  var clientAuthEnabled = false;
}

/**
 * Chat Widget configuration settings
 * Depending on the type of channel connecting to, the mandatory requirements
 * in the settings object are different.
 *
 * If the channel has client authentication enabled, the settings must pass
 * clientAuthEnabled: true
 *
 * If the channel has client authentication disabled, the settings should pass
 * channelId, and optionally, userId.
 */
var chatWidgetSettings;
if (clientAuthEnabled) {
  chatWidgetSettings = {
    URI: '<url>',
    clientAuthEnabled: true
  };
} else {
  chatWidgetSettings = {
    URI: '<your-instance-url>',
    channelId: '<Your-channel-id>',
    botButtonIcon: 'images/edward.png',
    botIcon: 'images/edward.png',
    logoIcon: 'images/edward.png',
    enableBotAudioResponse: true,
    "i18n": {
      "en": {
        "chatTitle": "Edward"
      }
    },
    enableClearMessage: "true"
```

```

};
}

/**
 * Function to generate JWT tokens. It returns a Promise to provide tokens.
 * The function is passed to SDK which uses it to fetch token whenever it needs
 * to establish connections to chat server
 * @returns {Promise} Promise to provide a signed JWT token
 */
var generateToken = function() {
  return new Promise((resolve) => {
    mockApiCall('https://mockurl').then((token) => {
      resolve(token);
    });
  });
}

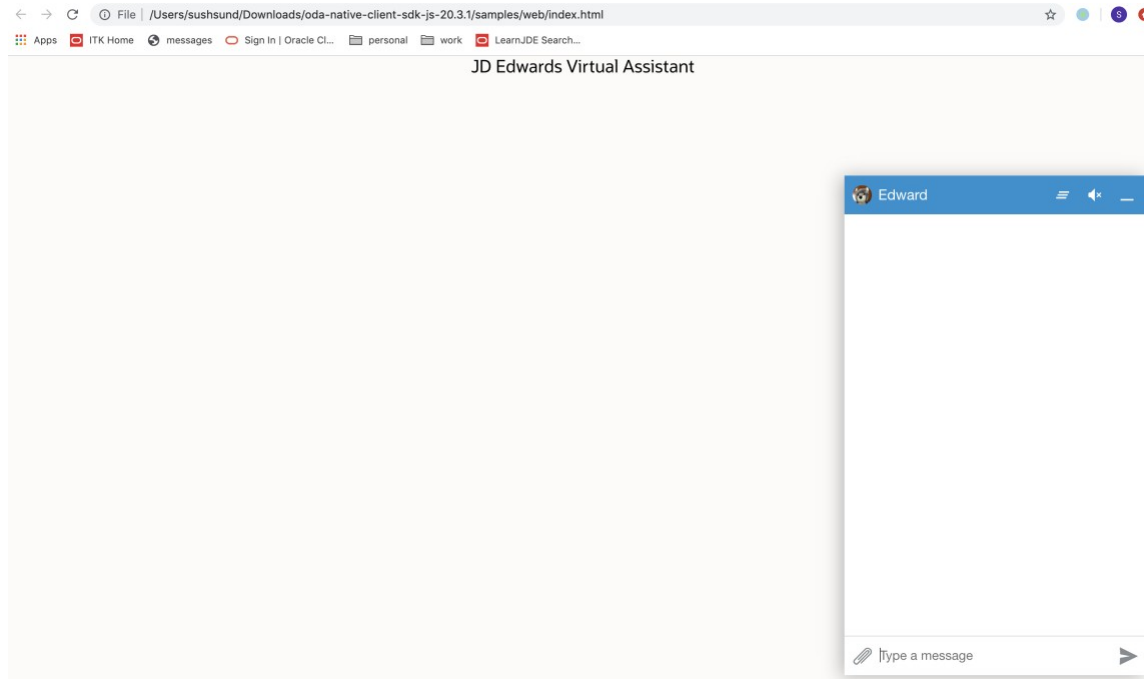
/**
 * A function showing an endpoint call to backend api to provide authentication token
 * The recommended behavior is fetching the token from the back-end server
 * @returns {Promise} Promise to provide a signed JWT token
 */
var mockApiCall = function() {
  return new Promise((resolve) => {
    var now = Math.floor(Date.now() / 1000)
    var payload = {
      iat: now,
      exp: now + 3600,
      channelId: '<channel-id>',
      userId: 'abc'
    };
    var SECRET = '<channel-secret>';

    // An unimplemented function generating signed JWT token with given header, payload, and signature
    var token = generateToken({ alg: 'HS256', typ: 'JWT' }, payload, SECRET);
    resolve(token);
  });
}

```

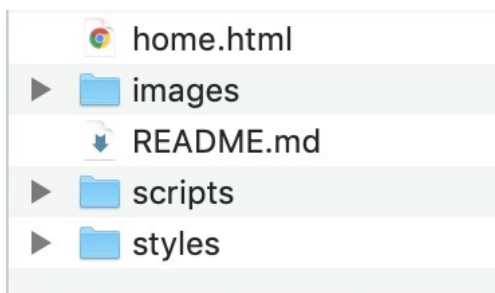
To modify the original settings.js file:

- In settings.js, replace “<your-instance-url>” with the URL of your ODA instance without the https part and replace “<your-channel-id>” with the channel ID created for your Skill in the previous section.
- Navigate to the web directory and click home.html to open the file in the browser.
The chat interface will be initialized and connected to your application. Click the chat button at the bottom right of the window to open the chat interface.



Uploading a Chatbot as an E1 Page

1. Navigate to the directory “oda-native-client-sdk-js-20.3.x\samples \web” and compress the contents of the folder as an archive, for example, archive.zip.



2. In Server Manager Console, edit the JAS ini settings under Security by making the following changes::
 - a. Add all the required file extension types present in the compressed folder, archive.zip, to the existing “E1Page Content File-Extension List” for the successful upload of the E1 Page. In this case, you have to add “md” as you have readme.md file in your compressed folder, archive.zip. If you do not want to make any changes to this list, you can remove the readme.md file from the compressed folder before uploading it as an E1 Page.
E1Page Content File-Extension List: asp,bmp,css,dat,dip,gif,htm,html,ico,img,jfif,jpe,jpeg,jpg,js,mf,pdf,png,svg,tif,tiff,xml,**md**
 - b. Setting for Strict Checking of File Upload and Download: Disabled
 - c. Setting for Strict File Extension Check: Disabled
3. Restart JAS after making the changes.

Upload Inclusion Lists [Return To Top](#)

These settings define the file extensions that will be allowed when uploading files to the server. Different features may have different lists of valid extensions. This is a security feature to help prevent users from uploading files to the server that could potentially be harmful to the system.

Default Extension List [?](#) [?](#)

E1Page Content File-Extension List [?](#) [?](#)

Media Object Extension List [?](#) [?](#)

Setting for Strict Checking of File Upload and Download [?](#) ☐ [?](#)

Setting for Strict File Extension Check [?](#) ☐ [?](#)

[Revert to Defaults](#) [Apply](#)

- Log in to JAS and upload the compressed contents as an E1 Page. For this step, select Manage Content, Classic Pages. The E1 Page can now be included as part of a Composed Page.

JDE_Admin_Bot EnterpriseOne JDEDemoE1Page Health & Safety Incidents Work With OVFS Defining OVFS Incident Self Servi >

JD Edwards Virtual Assistant

E1Page Manager

Name [?](#)

Available Languages [?](#)

Product Code Reserved for Clients

Description

Select Page Type:
☐ Page URL ☒ Upload HTML Content

Upload HTML Content:
 Choose file [Upload](#) [View Content](#) [Download](#)

JDE_Admin_Bot EnterpriseOne JDEDemoE1Page Health & Safety Incidents Work With OVFS Defining OVFS Incident Self Service Team Backorders Team Held Orders Team Late >

JD Edwards Virtual Assistant

Edward

Type a message [➤](#)

CONNECT WITH US

1. Call +1.800.ORACLE1 or visit oracle.com.
Outside North America, find your local office at oracle.com/contact.



blogs.oracle.com



facebook.com/oracle



twitter.com/oracle

Copyright © 2020, Oracle and/or its affiliates. All rights reserved. This document is provided for information purposes only, and the contents hereof are subject to change without notice. This document is not warranted to be error-free, nor subject to any other warranties or conditions, whether expressed orally or implied in law, including implied warranties and conditions of merchantability or fitness for a particular purpose. We specifically disclaim any liability with respect to this document, and no contractual obligations are formed either directly or indirectly by this document. This document may not be reproduced or transmitted in any form or by any means, electronic or mechanical, for any purpose, without our prior written permission.

This device has not been authorized as required by the rules of the Federal Communications Commission. This device is not, and may not be, offered for sale or lease, or sold or leased, until authorization is obtained.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Xeon are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Opteron, the AMD logo, and the AMD Opteron logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group. 0120

Integrated Chatbot User Experience for the JD Edwards Administration

May 2020

Author: Sushmitha Sundar

